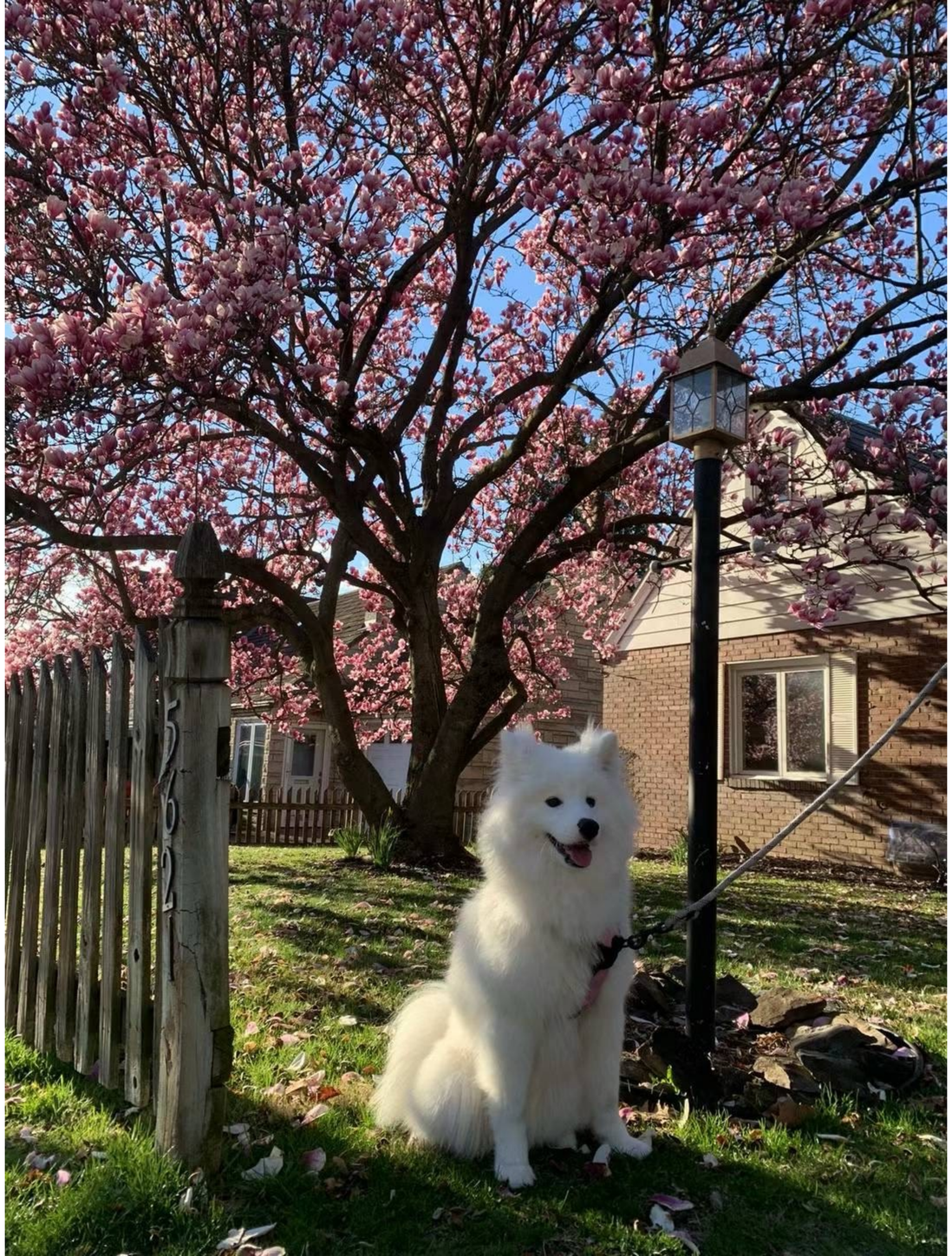
Visual Servoing

Zackory Erickson

Carnegie Mellon University
The Robotics Institute

16-762: Mobile Manipulation
Spring 2026


5602

Overview

Visual servoing motivation

Classifications

Perspective projection

Position-based visual servoing

Image-based visual servoing

Acknowledgment

Several slides from this lecture are derived from Charlie Kemp's course on Mobile Manipulation.

Charlie Kemp
(CTO at Hello Robot)
(Former Prof. at GT)

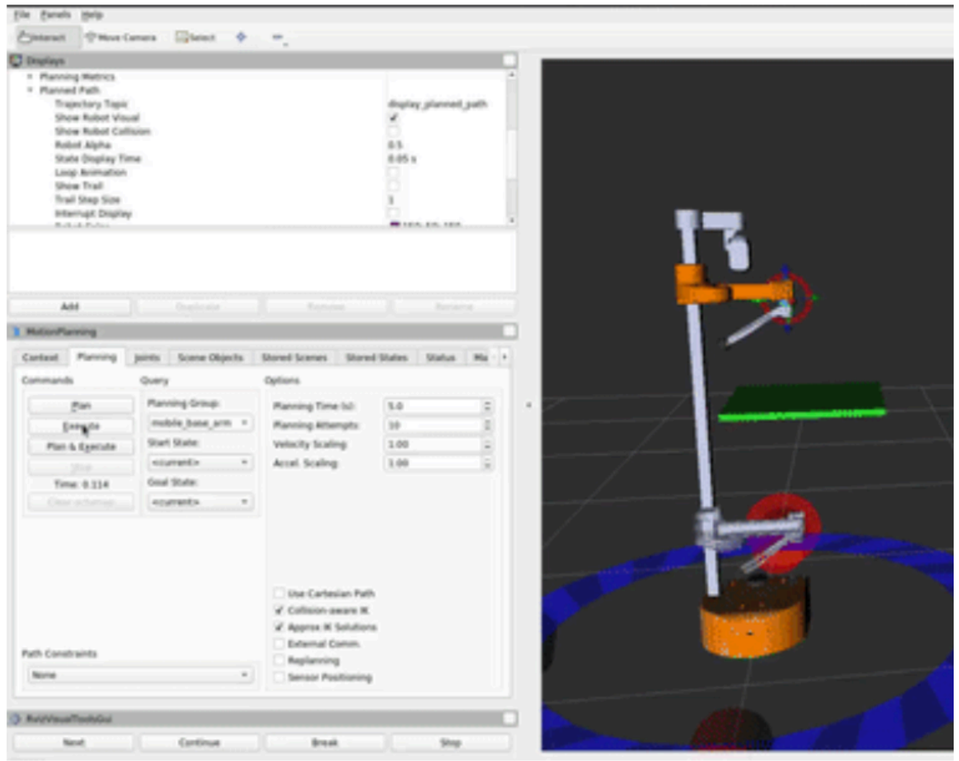
Find the CMU Mug

Manipulate the CMU Mug

Motion planning

(search for an entire trajectory)

- Global solutions
- Drawbacks
  - Difficult to move from simulation to the real world
  - Sensing rarely included
  - Usually assume known state
  - Usually assume well-modeled transitions between states
  - May not meet real-time constraints



Can we find an efficient and robust local solution?

The Perception-Action Loop

Traditional Pipeline:

- Sense: PointCloud → Segmentation → Pose Estimation (T_{obj}^{cam}).
- Plan: Inverse Kinematics (T_{base}^{ee}).
- Act: Trajectory execution.

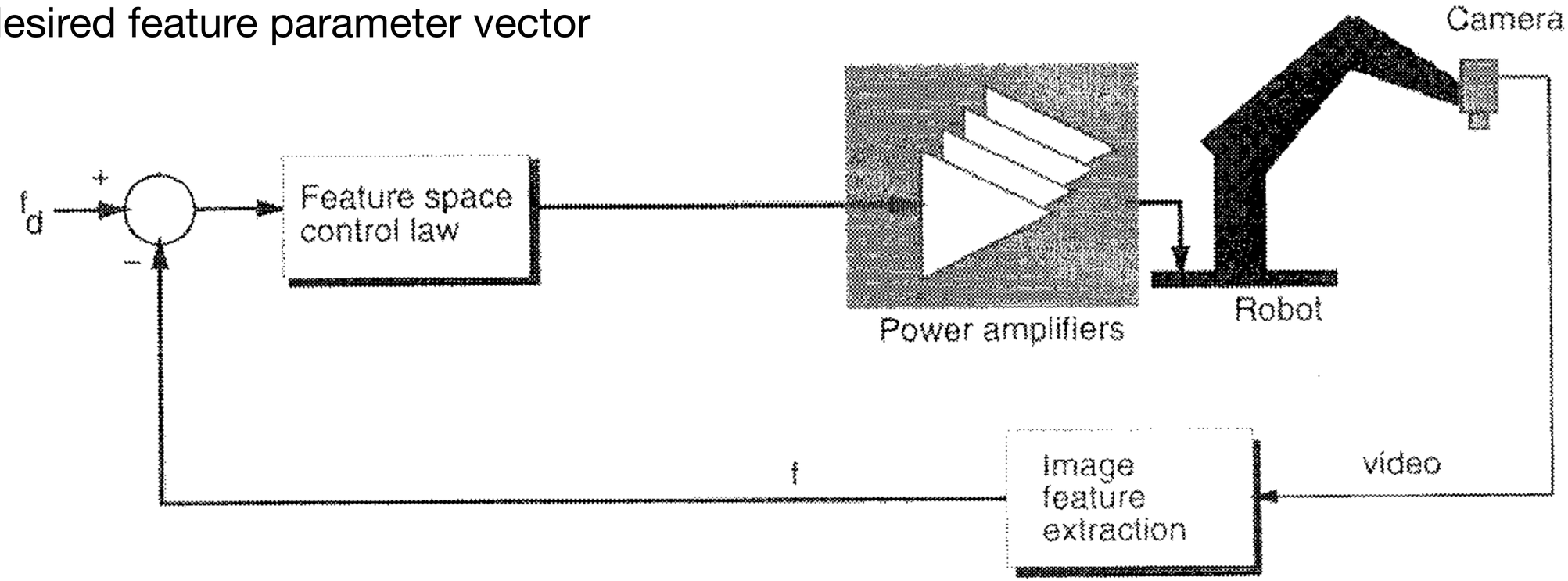
Visual Servoing Pipeline:

- Sense: Error in Image Plane $e = f - f_d$, where $f = [u, v]$ are image coordinates (i.e. pixel) and f_d is the desired coordinate position.
- Control: Velocity Command $\dot{q}$ as a function of the error e.

Why Servo? Directly eliminates calibration errors between camera and end-effector.

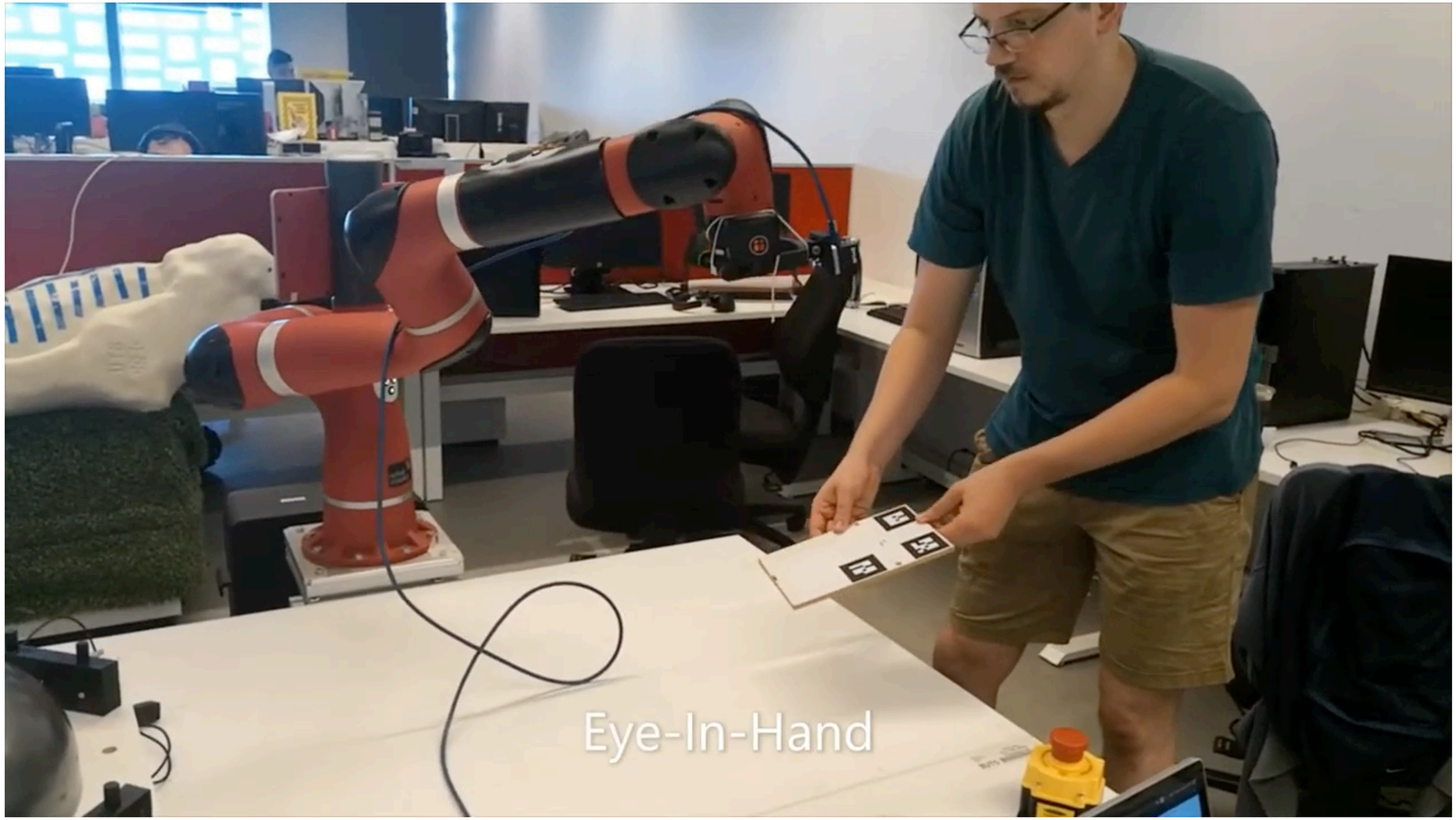
Visual Servoing

f_d = desired feature parameter vector



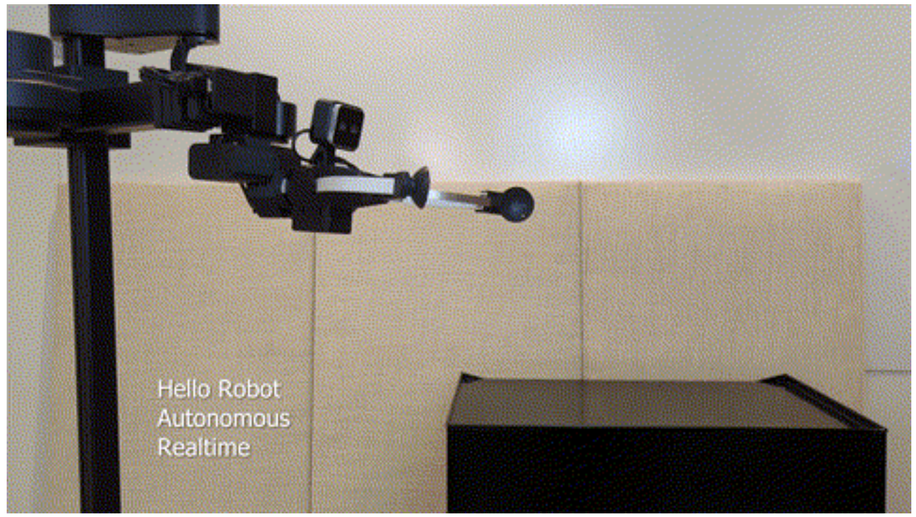
S. Hutchinson, G. Hager, and P. Corke, **A tutorial on visual servo control**, *IEEE Transactions on Robotics and Automation*, vol. 12, pp. 651–670, October 1996.

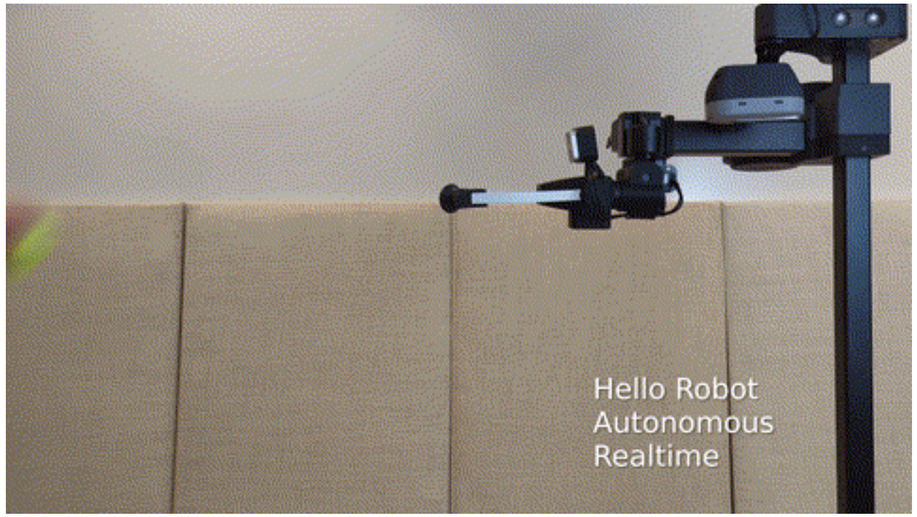
Visual Servoing

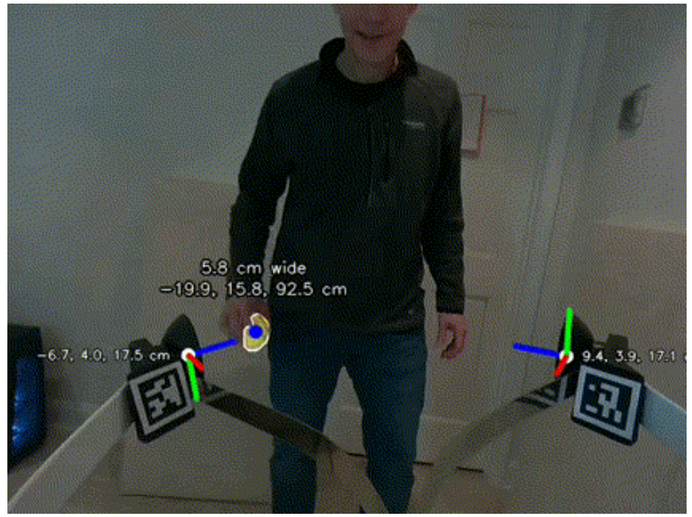


https://www.youtube.com/watch?v=cl6wHxglQvc

Visual Servoing on Stretch







https://github.com/hello-robot/stretch_visual_servoing

Visual Servoing Classifications

Eye-to-Hand: Camera fixed in world (or on robot head looking at arm).

- Example: Stretch 3 head camera (D435i) looking at workspace.

Eye-in-Hand: Camera attached to end-effector.

- Example: Stretch 3 gripper camera (D405).

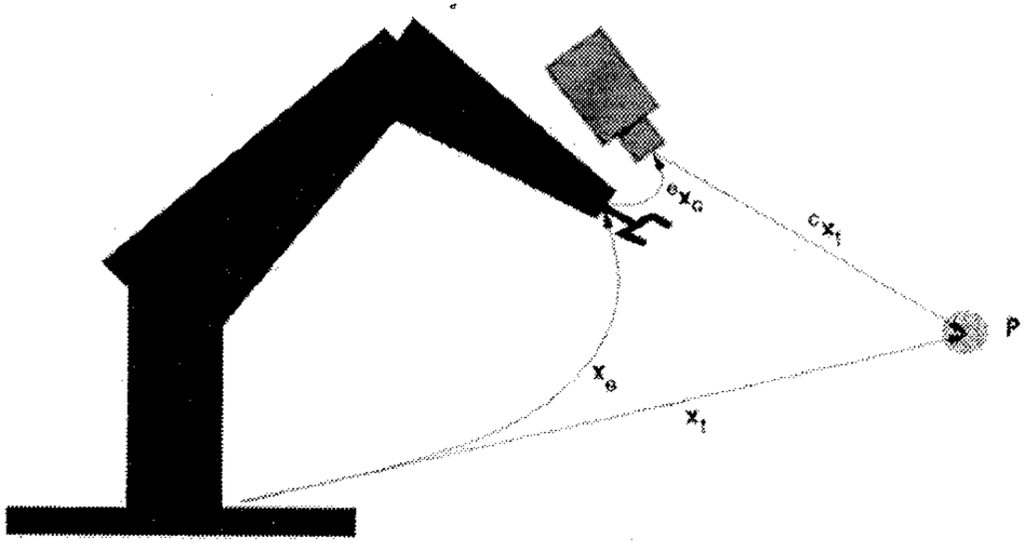
PBVS (Position-Based Visual Servoing):

- Reconstructs 3D pose of object. Servos in 3D space.
- Sensitive to calibration.

IBVS (Image-Based Visual Servoing):

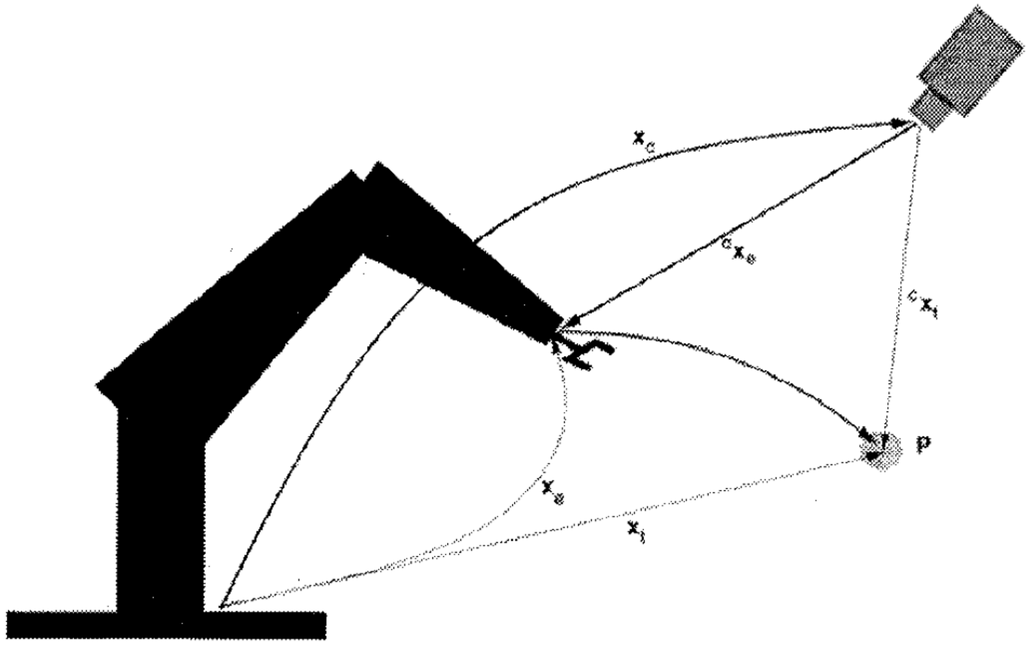
- Servos directly in 2D pixel coordinates.
- Robust to calibration errors; ideal for “final inch” grasping.

Eye-in-Hand vs Fixed Camera



End effector mounted

Eye-in-Hand

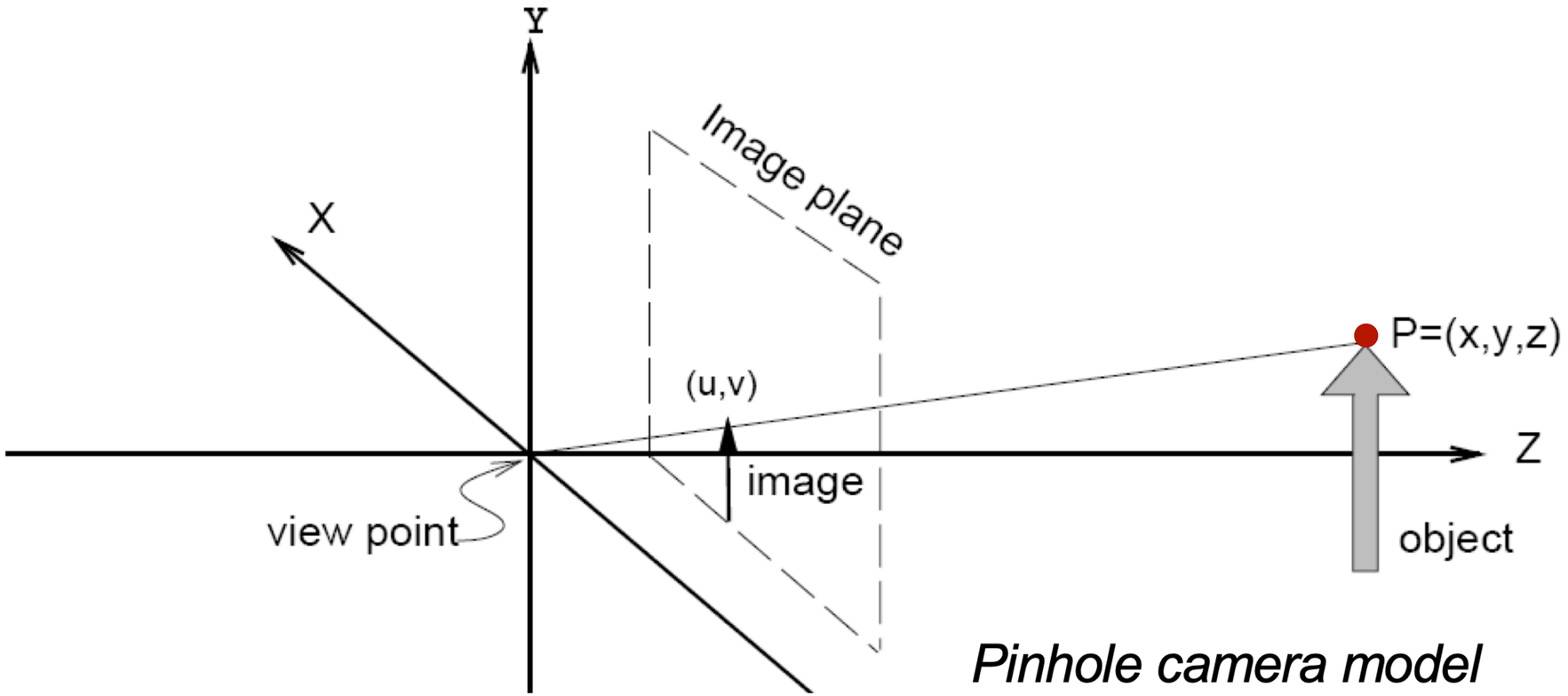


Fixed camera

Eye-to-Hand

S. Hutchinson, G. Hager, and P. Corke, A tutorial on visual servo control, *IEEE Transactions on Robotics and Automation*, vol. 12, pp. 651–670, October 1996.

Perspective Projection



Pinhole camera model

$$\pi(x, y, z) = \begin{bmatrix} u \\ v \end{bmatrix} = \frac{\lambda}{z} \begin{bmatrix} x \\ y \end{bmatrix}$$

λ = focal length

Perspective Projection

$$\pi(x, y, z) = \begin{bmatrix} u \\ v \end{bmatrix} = \frac{\lambda}{z} \begin{bmatrix} x \\ y \end{bmatrix}$$

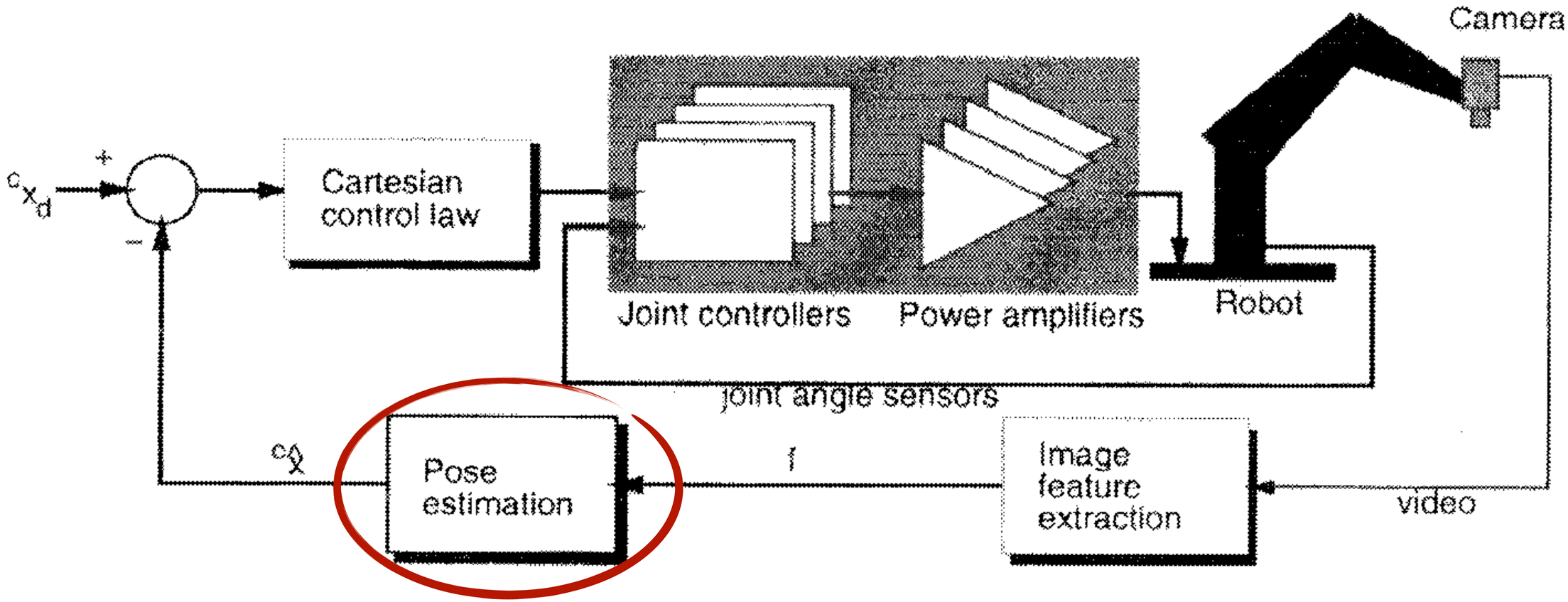
For a point (x, y, z) on an object, a corresponding point in the image plane is $u = \frac{\lambda}{z} x$ and $v = \frac{\lambda}{z} y$.

Note: If P is defined in a different coordinate frame, it must first be transformed to the camera frame.

Camera Calibration

- Estimates parameters
  - focal length, principal point, skew coefficient, distortions (radial and tangential)
- Rectify the images

Position-based Visual Servoing



- Benefit: Don’t need to deal with image transforms or image Jacobians.

Position-based Visual Servoing

(with a fixed camera)

End effector pose

Fixed goal pose

Kinematic error function: $$E_{pp}(x_e, S, {}^eP) = x_e({}^eP) - S$$

Proportional gain

Transform goal to robot frame

Proportional control law: $$u = -k(\hat{x}_e({}^eP) - \hat{x}_c({}^c\hat{S}))$$

Position-based Visual Servoing

(camera mounted to end effector)

End effector pose

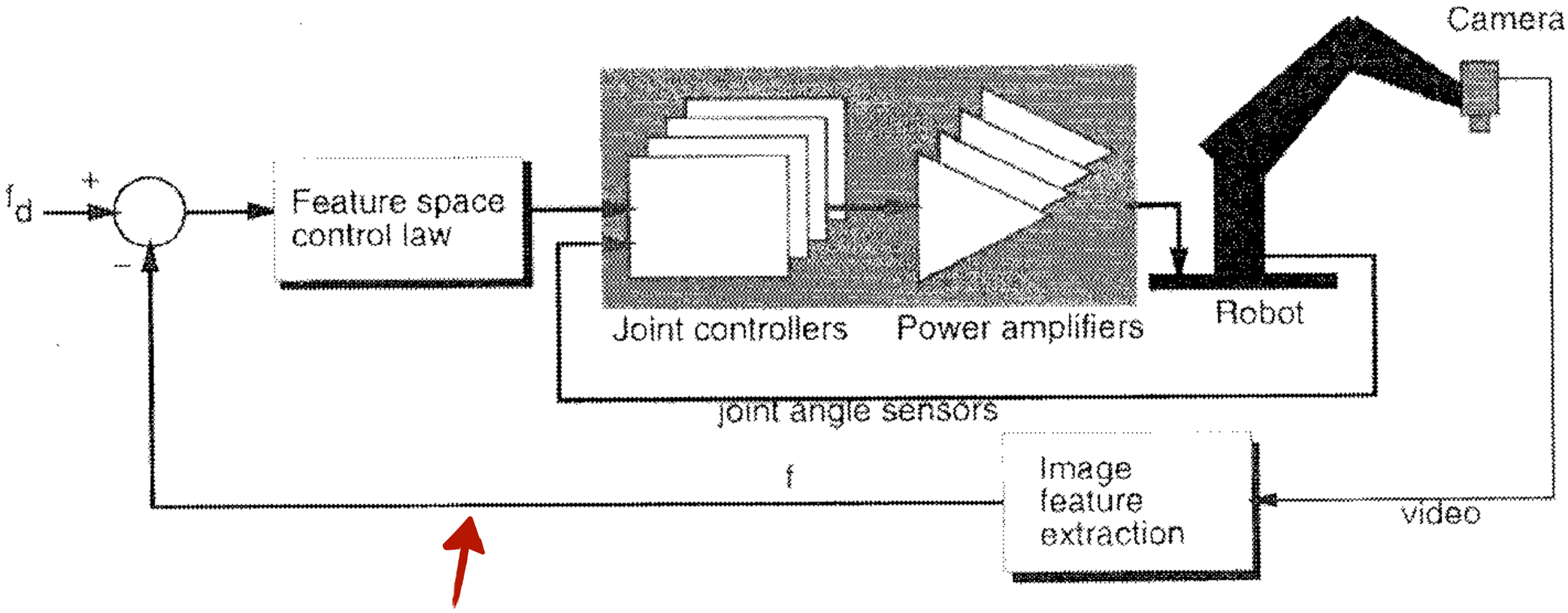
Goal pose transformed to end-effector frame

Kinematic error function: $$E_{pp}(x_e, S, {}^eP) = {}^eP - {}^ex_0(S)$$

Transform camera calibrated goal to end effector frame

Proportional control law: $$u = -k({}^eP - {}^e\hat{x}_c({}^c\hat{S}))$$

Image-based Visual Servoing



No more pose estimation

S. Hutchinson, G. Hager, and P. Corke, A tutorial on visual servo control, *IEEE Transactions on Robotics and Automation*, vol. 12, pp. 651–670, October 1996.

Image Jacobian

$$\dot{f} = \begin{bmatrix} \dot{u} \\ \dot{v} \end{bmatrix} = J_v(r)\dot{r}$$

Where $\dot{r}$ represents the end effector velocity

$$J_v(r) = \left[\frac{\delta \mathbf{f}}{\delta \mathbf{r}}\right] = \begin{bmatrix} \frac{\delta f_1(\mathbf{r})}{\delta r_1} & \cdots & \frac{\delta f_1(\mathbf{r})}{\delta r_m} \\ \vdots & & \vdots \\ \frac{\delta f_k(\mathbf{r})}{\delta r_1} & \cdots & \frac{\delta f_k(\mathbf{r})}{\delta r_m} \end{bmatrix}$$

Partial derivatives of image coordinates wrt. end effector pose

S. Hutchinson, G. Hager, and P. Corke, A tutorial on visual servo control, *IEEE Transactions on Robotics and Automation*, vol. 12, pp. 651–670, October 1996.

Example Image Jacobian

Translational and rotational velocity of end effector in camera frame

Jacobian

$$\begin{bmatrix} \dot{u} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} \frac{\lambda}{z} & 0 & -\frac{u}{z} & -\frac{uv}{\lambda} & \frac{\lambda^2+u^2}{\lambda} & -v \\ 0 & \frac{\lambda}{z} & -\frac{v}{z} & \frac{-\lambda^2-v^2}{\lambda} & \frac{uv}{\lambda} & u \end{bmatrix} \begin{bmatrix} T_x \\ T_y \\ T_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

S. Hutchinson, G. Hager, and P. Corke, A tutorial on visual servo control, *IEEE Transactions on Robotics and Automation*, vol. 12, pp. 651–670, October 1996.

Inverting the Image Jacobian

$$\dot{r} = J_v^+(r)\,\dot{f}$$

Our error function

$$e(f) = f - f_d$$

Control input defined as end effector velocity

$$u = J_v^+(r)\,\dot{f}$$

Proportional control law

$$u = KJ_v^+(r)e(f)$$

S. Hutchinson, G. Hager, and P. Corke, A tutorial on visual servo control, *IEEE Transactions on Robotics and Automation*, vol. 12, pp. 651–670, October 1996.

Servoing without calibration

- Estimate the Jacobian online

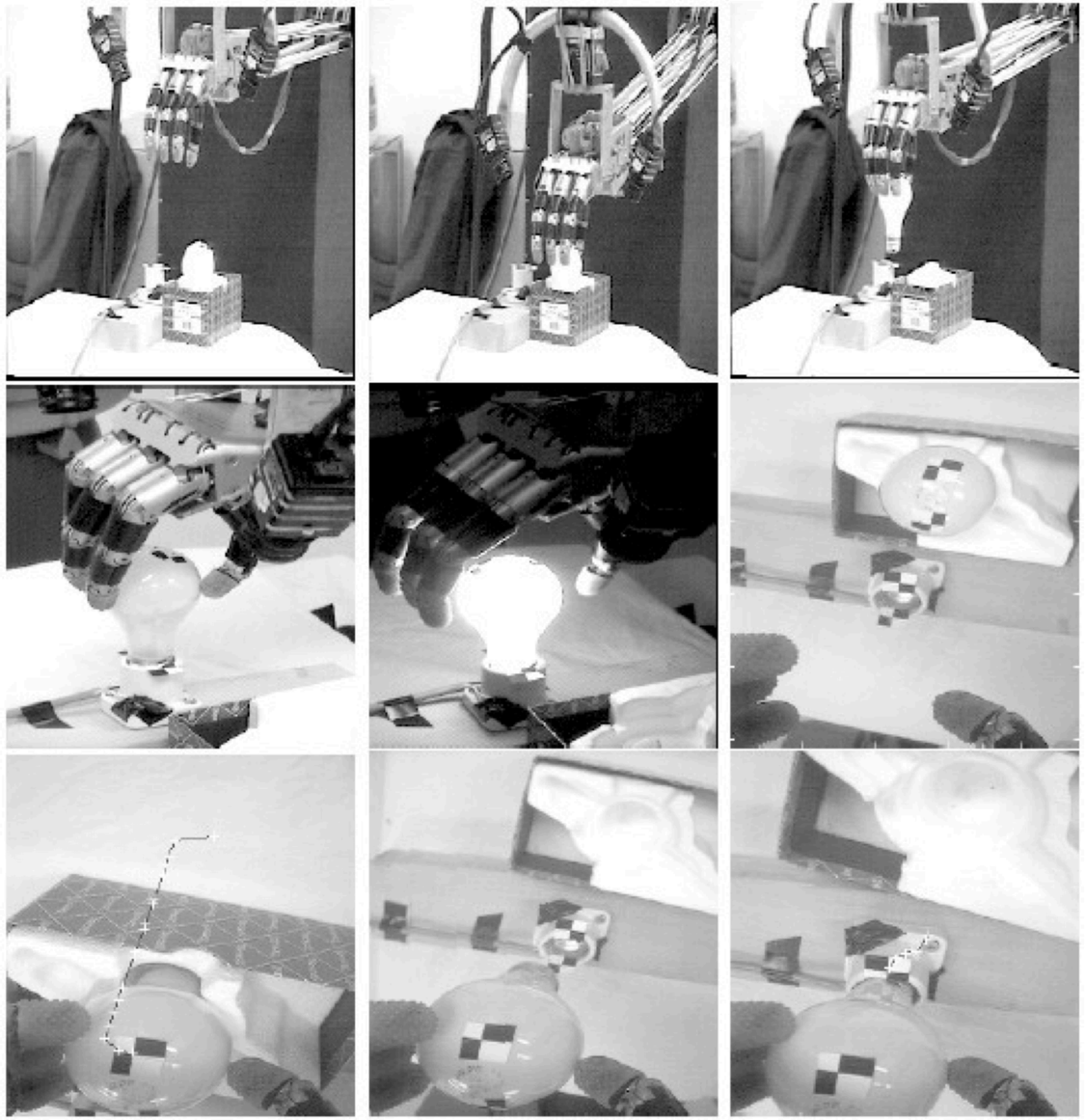
Estimation of the full Jacobian was solved mathematically by Broyden in the 60's [20], and later rediscovered in robotics by [11, 10, 15]. A first order updating formula which converges to the Jacobian after n linearly independent moves is:

$$\hat{J}_{k+1} = \hat{J}_k + \frac{(\Delta \mathbf{y}_{measured} - \hat{J}_k \mathbf{x})\mathbf{x}^T}{\mathbf{x}^T \mathbf{x}}$$

Note that this estimation accepts movements along arbitrary directions $\mathbf{x}$ and thus needs no additional data other than what is available as a part of the manipulation task we want to solve.

Jägersand M. Nelson R. Visual Space Task Specification, Planning and Control. In Proc. of IEEE Int. Symp. on Computer Vision 95, p 521-526, 1995.

Dexterous manipulation without calibration



Jägersand M. Nelson R.
Visual Space Task Specification, Planning and Control. In Proc. of IEEE Int. Symp. on Computer Vision 95, p 521-526, 1995.

Discussion questions?

- When is visual servoing a good idea?
- Are there certain problems visual servoing could solve consistently?
- How should we deal with a mobile base?
- When is visual servoing a bad idea?

What would you do?